# EN4554 — Assignment 2

Convolutional Neural Networks

A plain-English companion to my notebook

**Lakshan R.G.R**

Index 220355M

What each question asked · what my code does

the mathematics behind it · what the results mean

# What is in here

This document is a companion to `EN4554_a02.ipynb`. For every part of the assignment it answers four questions, in the same order each time:

| Section | What it tells you |
|---|---|
| **The question** | What the assignment actually asked for, in plain words |
| **The idea** | The concept behind it, with no jargon on first use |
| **The mathematics** | Every equation the code implements, and what each symbol means |
| **The code** | Which notebook cell does it, and what that cell is doing |
| **The result** | The numbers I measured and what they mean |


1. **The big picture** — how the four tasks fit together
    1. The one-sentence story
    2. Map of the notebook
2. **Task 1 — Backpropagation by hand**
    1. Three derivatives: Linear, ReLU, softmax cross-entropy
    2. Proving the gradients are right
    3. Training the MLP — and why it stops at 50%
    4. Why deep sigmoid networks fail
3. **Task 2 — Optimizers**
    1. SGD, momentum and Adam as formulas
    2. Why the narrow-valley picture matters
    3. Learning-rate sensitivity
4. **Task 3 — Regularization and shortcuts**
    1. Six ways to stop a network memorising
    2. The degradation problem and the residual fix
5. **Task 4 — The best network under 500k parameters**
6. **Explaining this to someone in five minutes**
7. **Symbol glossary**

# 1. The big picture

## 1.1 The one-sentence story

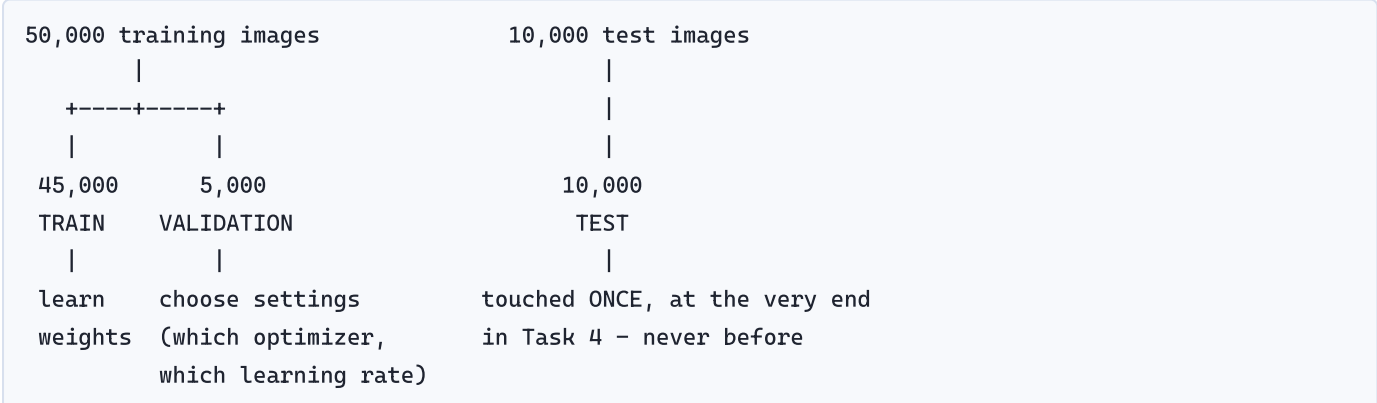
> **THE THREAD RUNNING THROUGH ALL FOUR TASKS**
>
> **Structure beats size.** A network that is *built* to understand images needs far fewer numbers than one that must learn everything from scratch — and the assignment walks through that idea one piece at a time.

Each task adds one piece:

| Task | The question it answers | The answer it gives |
|---|---|---|
| **1** | How does a network learn at all? | By backpropagation. I wrote it by hand, with no autograd, and proved it correct. A plain fully-connected network tops out near **50%** on CIFAR-10. |
| **2** | Does it matter *how* we step downhill? | Enormously. The same network scores anywhere from **43%** to **80%** depending only on the optimizer and learning rate. |
| **3** | Why do networks fail, and how do we fix them? | Two different failures. Too little data → it memorises (fix: regularization). Too many layers → it cannot be trained (fix: shortcut connections). |
| **4** | Put it together under a budget. | Best network I can build with at most 500,000 numbers and 20 passes over the data. |

## 1.2 The data, once, for the whole assignment

**CIFAR-10** is 60,000 colour photographs, each a tiny 32×32 pixels, belonging to 10 classes (aeroplane, car, bird, cat, deer, dog, frog, horse, ship, truck). Each image is $32 \times 32 \times 3 = 3072$ numbers — three colour channels per pixel.




> **WHY THREE SPLITS AND NOT TWO**
>
> If I pick my settings by looking at a set of images, then those images have secretly helped train the model — my score on them is flattering, not honest. So the **validation** set takes all the decisions, and the **test** set stays sealed until there is nothing left to decide. That is why the notebook only reads the test set in one cell, right at the end, and caches the answer so it cannot accidentally be measured twice.

One more setup detail that matters. Every image is **normalised**: for each colour channel I subtract the mean and divide by the standard deviation,

$$x_{\text{normalised}} = \frac{x - \mu_{\text{channel}}}{\sigma_{\text{channel}}}$$

with $\mu$ and $\sigma$ computed from the **45,000 training images only**. Using all 50,000 would leak information about the validation images into the model. The measured values were $\mu = (0.4912,\ 0.4820,\ 0.4464)$ and $\sigma = (0.2472,\ 0.2436,\ 0.2617)$.

> **IF SOMEONE ASKS "WHY NORMALISE?"**
> Raw pixels run 0–255. Weights get updated by a step proportional to the input size, so huge inputs mean huge, unstable steps. Centring the data near zero with a spread of about 1 makes every input comparable and lets one learning rate work for all of them.

# 2. Task 1 — Backpropagation by hand

> **WHAT THE ASSIGNMENT ASKED**
> **(a)** Implement `Linear`, `ReLU` and softmax cross-entropy with explicit forward/backward — *no autograd* — and assemble an MLP.
> **(b)** Verify the gradients against autograd and against centred finite differences; report the relative errors.
> **(c)** Train it with SGD on flattened CIFAR-10; plot the loss and report validation accuracy.
> **(d)** For a 10-layer MLP, plot the per-layer gradient norm at initialisation with sigmoid and with ReLU. Explain.

## 2.1 The idea: learning is just blame assignment

A network makes a guess, and the guess is wrong by some amount — the **loss**. Every weight in the network shares some of the blame. **Backpropagation** is the procedure that works out exactly how much blame each weight deserves, and it does so by walking *backwards* through the network applying the chain rule.

```
FORWARD   image --> Linear --> ReLU --> Linear --> ReLU --> Linear --> loss
                                                                       |
BACKWARD   dL/dW1 <-- dL/dx <-- dL/dx <-- dL/dx <-- dL/dx <-- dL/dz <---+
           "how much is each weight to blame?"
```

Each layer only has to know two things: how to compute its output from its input (**forward**), and how to turn "blame on my output" into "blame on my input and on my own weights" (**backward**). Chain those together and the whole network trains.

## 2.2 The mathematics — three derivatives

Throughout, $g$ means the gradient arriving from the layer above, i.e. $g = \dfrac{\partial L}{\partial (\text{this layer's output})}$.

### (i) The Linear layer

Forward, for a batch of $N$ images: $y = xW + b$ with $x \in \mathbb{R}^{N \times d_{in}}$, $W \in \mathbb{R}^{d_{in} \times d_{out}}$, $b \in \mathbb{R}^{d_{out}}$.

$$\frac{\partial L}{\partial x} = g W^{\top} \qquad \frac{\partial L}{\partial W} = x^{\top} g \qquad \frac{\partial L}{\partial b} = \sum_{n=1}^{N} g_n$$

> **HOW TO REMEMBER IT**
> To differentiate a matrix product, **multiply by the other factor, transposed**. And the shapes only fit together one way — if your dimensions match, you almost certainly have the right formula. The bias was added to every row of the batch, so its gradient is the **sum down the rows**.

### (ii) ReLU

Forward: $y = \max(x, 0)$.

$$\frac{\partial L}{\partial x} = g \odot \mathbb{1}[x > 0]$$

($\odot$ means multiply element by element; $\mathbb{1}[\cdot]$ is 1 where the condition holds and 0 elsewhere.)

> **SAY IT IN ONE LINE**
> ReLU is a gate. Where it let the signal *through* going forwards, it lets the blame through coming backwards. Where it *blocked* the signal, it blocks the blame.

### (iii) Softmax + cross-entropy, fused into one block

**Softmax** turns raw scores (called **logits**) into probabilities that add to 1. **Cross-entropy** then measures how surprised we are by the right answer. For logits $z \in \mathbb{R}^{N \times C}$ and true labels $y$:

$$L = \frac{1}{N} \sum_{n=1}^{N} \left( \log \sum_{c=1}^{C} e^{z_{nc}} \; - \; z_{ny_n} \right)$$

And here is the result worth remembering, because it is beautiful:

$$\boxed{\frac{\partial L}{\partial z} = \frac{1}{N} \Big( \mathrm{softmax}(z) - \mathrm{onehot}(y) \Big)}$$

> **IN WORDS: PREDICTED PROBABILITY MINUS TRUE PROBABILITY**
> If the model says 0.7 for the correct class, the gradient on that logit is $-0.3$ — push it up. If it says 0.6 for a wrong class, the gradient is $+0.6$ — push it down. Every messy term of the softmax derivative cancels against the logarithm in cross-entropy. **This is why the two are always written as one block**, and it is the single most likely thing to be asked about in an exam.

> **THE TRAP: OVERFLOW**
> $e^z$ becomes `inf` for $z$ above about 88 in single precision, and the loss turns into `nan`. The fix is to subtract the largest logit in each row first: $z \leftarrow z - \max_c z_c$. The softmax is unchanged (the factor cancels top and bottom) but now every exponent is $\leq 0$. My code does this on the first line of the forward pass.

## 2.3 The code — what each cell does

| Cell | What it defines | What it is doing |
|---|---|---|
| `code 4` | `Linear`, `ReLU`, `Sigmoid`, `SoftmaxCrossEntropy` | The four building blocks. Each stores whatever its backward pass will need during `forward` — `Linear` keeps `self.x`, `ReLU` keeps a boolean mask, `Sigmoid` keeps its *output* (because $\sigma' = \sigma(1-\sigma)$). |
| `code 5` | `MLP`, `SGDManual` | `MLP.backward` is the whole trick in three lines: walk the layer list **in reverse**, handing each layer's output gradient to the layer below. |
| `code 6` | `check_against_autograd` | Builds the same network with `nn.Linear`, copies my weights in, compares. |
| `code 7` | `check_finite_differences` | Nudges single weights up and down and measures the slope directly. |
| `code 8–9` | training + plot | Trains 3072→512→256→10 with my own SGD. |
| `code 10–11` | `layer_grad_norms` + plot | One forward/backward pass through a 10-layer net, recording gradient size per layer. |

The core of it, in about ten lines:

```
class Linear:
    def forward(self, x):
        self.x = x                  # remember: needed for dL/dW
        return x @ self.W + self.b

    def backward(self, g):
        self.dW = self.x.T @ g      # (in,N)@(N,out) -> (in,out)
        self.db = g.sum(dim=0)      # bias was added to every row
        return g @ self.W.T       # pass blame to the layer below

class MLP:
    def backward(self, g):
        for layer in reversed(self.layers):   # <-- the entire idea
            g = layer.backward(g)
```

## 2.4 Part (b) — proving the gradients are right

Two independent checks, which would fail for different reasons, so passing both is strong evidence.

**Check 1 — against autograd.** Rebuild the identical network with PyTorch's own `nn.Linear` layers, copy my weights across, and compare.

> **ONE DETAIL THAT CATCHES PEOPLE OUT**
>
> PyTorch stores a linear layer's weight as `(out_features, in_features)`. Mine is `(in, out)` — the other way round. So both the copy and the comparison need a **transpose**. Get this wrong and you will think your gradients are broken when they are fine.

**Check 2 — centred finite differences.** This trusts nothing at all, not even PyTorch. Nudge one weight by a tiny $\varepsilon$ in each direction and measure the slope:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta+\varepsilon) - L(\theta-\varepsilon)}{2\varepsilon}$$

I use the *centred* version rather than the one-sided one because its error shrinks like $\mathcal{O}(\varepsilon^2)$ instead of $\mathcal{O}(\varepsilon)$. The number reported is the **relative error**:

$$\text{rel err} = \frac{\left|\text{analytic} - \text{numeric}\right|}{\left|\text{analytic}\right| + \left|\text{numeric}\right| + 10^{-12}}$$

### What I measured

Both checks report the **same quantity in the same layout**, so they read side by side:

| Reference method | Worst rel. error | Median rel. error | Verdict |
|---|---|---|---|
| autograd | $1.0 \times 10^{-11}$ | $2.5 \times 10^{-16}$ | match |
| centred finite differences | $3.7 \times 10^{-9}$ | $1.0 \times 10^{-10}$ | match |

**WHY AUTOGRAD'S *WORST* RELATIVE ERROR LOOKS LARGER THAN ITS MEDIAN**

The median relative error against autograd is $2.5 \times 10^{-16}$ — one unit in the last digit of a 64-bit number, i.e. identical. The worst case reaches $1.0 \times 10^{-11}$ only because a handful of gradient entries are themselves **near zero**, where a round-off-sized absolute difference is a large *fraction* of a tiny value. The largest **absolute** difference across every tensor is $3.3 \times 10^{-16}$, which confirms there is no real disagreement. For near-zero entries the median is the honest figure.

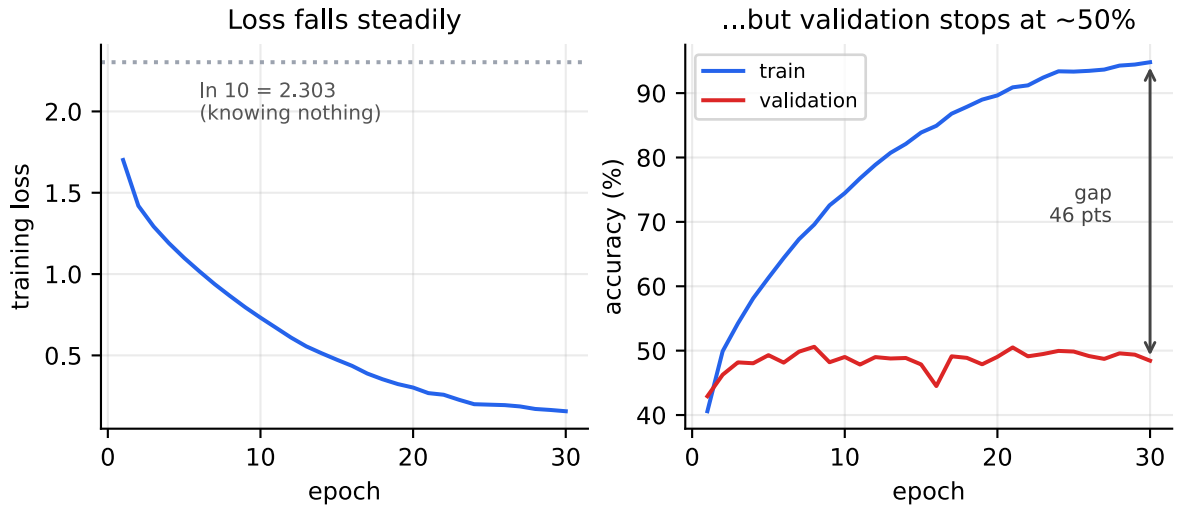
Finite differences sit higher, at $10^{-10}$–$10^{-9}$, and that is **expected rather than a defect**: the subtraction $L(\theta+\varepsilon) - L(\theta-\varepsilon)$ cancels most of the significant digits, setting a noise floor no implementation can beat.

**TWO THINGS THAT MAKE A CORRECT GRADIENT LOOK WRONG**

**1. Single precision.** Run the same check in `float32` and the relative error sits around $10^{-3}$, which looks like a bug. It is not — subtracting two nearly equal numbers destroys most of the digits. That is why the check runs in `float64`.

**2. ReLU's kink.** ReLU is not differentiable at exactly 0. If a value sits within $\varepsilon$ of zero, the two nudges land on opposite sides of the kink and that one weight reports a large error even though the code is right.

## 2.5 Part (c) — training it, and the 50% ceiling




**Task 1(c).** Left: the loss starts near $\ln 10 = 2.303$ — exactly what a network that knows nothing must output, since it can only assign $1/10$ to every class — and falls to 0.157. Right: training accuracy climbs to 94.8% while validation stalls near 50%. The arrow marks the 46-point gap.

| Quantity | Value |
|---|---|
| Network | 3072 → 512 → 256 → 10 |
| Parameters | 1,707,274 |
| Best validation accuracy | **50.6%** |
| Final training accuracy | 94.8% |
| Gap | 46 percentage points |

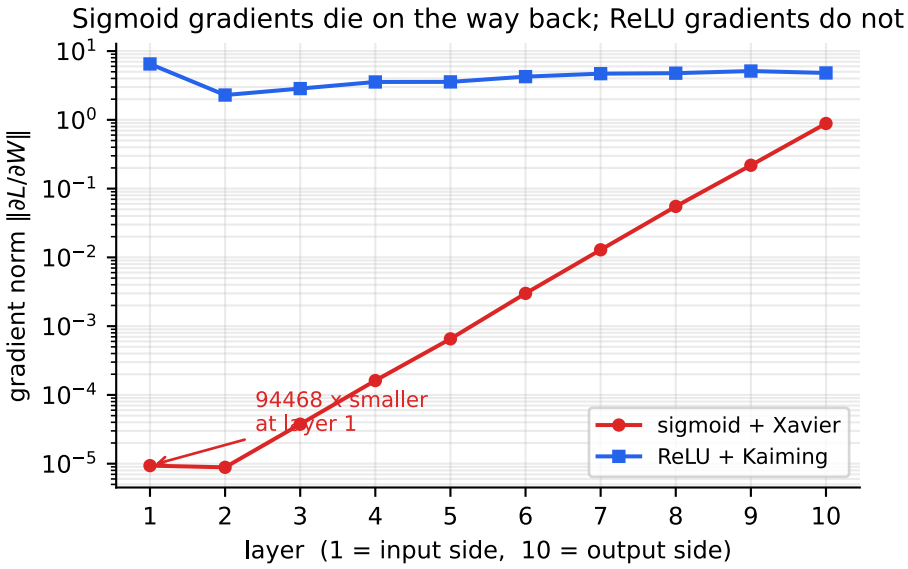
**THE IMPORTANT POINT IS THE CEILING, NOT THE GAP**

A fully-connected network cannot get much past 50–55% on CIFAR-10 **no matter how long you train it or how wide you make it**. This is a limit of the architecture, not a failure of the training. Three reasons:

- **Flattening destroys locality.** After `reshape`, the pixel at (5,5) and the pixel at (5,6) are just "input 165" and "input 166". The network is never told they are neighbours and must learn that separately for every pair.
- **No translation invariance.** A cat in the corner and the same cat shifted three pixels right are completely different input vectors. The network has to learn each position from scratch.
- **It is wasteful.** The first layer alone is $3072 \times 512 = 1{,}572{,}864$ weights, and it still only ever sees one fixed arrangement of pixels.

This is exactly the gap a convolution closes — and the small CNN in Task 2 reaches **80.4%** with **twelve times fewer parameters**.

## 2.6 Part (d) — why deep sigmoid networks will not train




**Task 1(d).** One forward and backward pass through a 10-layer network at initialisation, no training. The vertical axis is logarithmic. Sigmoid's gradient shrinks by a factor of about **94,000** on its way back to layer 1; ReLU's stays flat.

**Why sigmoid collapses.** The derivative of the sigmoid function is

$$\sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big) \;\le\; \tfrac{1}{4}$$

with the maximum $\frac{1}{4}$ reached only at $x = 0$. Backpropagation multiplies by one of these factors *per layer*. After 9 hidden layers the signal reaching the first layer is scaled by at most

$$\left(\tfrac{1}{4}\right)^{9} \approx 4 \times 10^{-6}$$

and in practice far less, because most units sit away from $x = 0$ where $\sigma'$ is much smaller than $\frac{1}{4}$. The early layers receive essentially no instruction and stay near their random starting values. This is the **vanishing gradient problem**.

**WHY RELU PLUS KAIMING INITIALISATION FIXES IT**

Two effects cancel exactly:

- ReLU's derivative is **0 or 1** — it never shrinks what passes through. But about half the units are off at initialisation, which costs roughly a factor of 2 in signal variance per layer.
- **Kaiming initialisation puts that factor back on purpose.** It sets $\mathbf{Var}[W] = \frac{2}{\textbf{fan-in}}$ rather than $\frac{1}{\textbf{fan-in}}$. **The 2 in that numerator *is* the ReLU correction.**

So the activation function and the initialiser are not independent choices — Kaiming was derived *for* ReLU. That is why the figure pairs sigmoid with Xavier ($\mathbf{Var}[W] = 1/\textbf{fan-in}$) and ReLU with Kaiming: it is the honest comparison.

**A DETAIL WORTH BEING ABLE TO EXPLAIN**

In the ReLU curve, layer 1 has the *largest* norm, which looks like it contradicts "flat". It does not. Layer 1's weight matrix is $3072 \times 128$ while every other hidden layer is $128 \times 128$ — 24 times more entries contributing to the norm. It is a size effect, not a gradient effect.

**THE 20-SECOND VERSION**

"Going backwards, each sigmoid layer multiplies the learning signal by at most a quarter. Ten layers of that and the first layer gets nothing — so it never learns. ReLU multiplies by 0 or 1 instead, and Kaiming initialisation compensates for the half of the neurons that are switched off, so the signal survives all the way back. That is why deep networks only started working around 2010."

# 3. Task 2 — Optimizers

**WHAT THE ASSIGNMENT ASKED**

**(a)** Implement SGD with momentum and Adam (with bias correction) as `torch.optim.Optimizer` subclasses. Show that they match `torch.optim.SGD` and `torch.optim.Adam`.
**(b)** On $f(x, y) = \frac{1}{2}(x^2 + 50y^2)$, plot the trajectories of SGD, SGD with momentum and Adam. Use `contour` to plot the contours. Explain the differences.
**(c)** Train a small CNN with each optimizer at three learning rates each. Plot validation accuracy versus learning rate and the training loss at the best rate. Discuss sensitivity.

## 3.1 The idea: the direction is obvious, the step size is not

Backpropagation tells us which way is downhill. It does *not* tell us how far to step, or whether to remember where we were going. That is the optimizer's job, and Task 1 used the most naive possible choice: step straight downhill, every time.

## 3.2 The mathematics — three update rules

$g_t$ is the gradient at step $t$, $\theta$ a parameter, $\eta$ the learning rate.

### Plain SGD

$$\theta_{t+1} = \theta_t - \eta\, g_t$$

Step downhill. Nothing is remembered between steps.

### SGD with momentum

$$b_t = \mu\, b_{t-1} + g_t \qquad\qquad \theta_{t+1} = \theta_t - \eta\, b_t$$

Instead of using the raw gradient, keep a running "velocity" $b$. Think of a ball rolling downhill rather than a walker taking independent steps.

**WHY MOMENTUM SPEEDS THINGS UP — THE KEY SUM**

If the gradient keeps pointing the same way, $b$ builds up as a geometric series:

$$1 + \mu + \mu^2 + \cdots = \frac{1}{1 - \mu}$$

With the usual $\mu = 0.9$ that is a **tenfold amplification** along directions that stay consistent. Directions that keep *flipping sign* cancel themselves out instead. So momentum accelerates steady progress and damps oscillation — which is exactly what the valley picture below shows.

**TWO WAYS OF WRITING MOMENTUM — BOTH CORRECT**

Textbooks often write the buffer as a weighted average, with a $(1-\mu)$ in front:

$$b_t = \mu\, b_{t-1} + (1-\mu)\, g_t \qquad \text{(EMA form)}$$

**This is the same algorithm.** The two buffers differ by exactly the constant factor $(1-\mu)$, which is absorbed into the learning rate: the EMA form at $\eta/(1-\mu)$ gives *numerically identical* parameters to the form above at $\eta$. I verified this in the notebook — they agree to $8 \times 10^{-17}$.

The only difference is what $\eta$ means. On a constant gradient the EMA form moves $\eta$ per step; PyTorch's form moves $\eta/(1-\mu) = 10\eta$ per step. I used PyTorch's form because part (a) requires matching `torch.optim.SGD`, whose default is this one.

**And note my Adam *does* use the $(1-\beta)$ form** — because Adam is defined that way, and its bias correction only makes sense for a weighted average whose weights sum to 1.

## Adam

Keep a running mean *and* a running variance of the gradient, then divide one by the square root of the other:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \qquad \theta_{t+1} = \theta_t - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

**WHAT THE DIVISION BUYS YOU**

Dividing by $\sqrt{\hat{v}}$ makes the step **scale-free**. Each parameter is normalised by its own recent gradient size, so the step is roughly $\eta$ whether that parameter's gradients are huge or tiny. That is what makes Adam forgiving about badly scaled problems — and, as part (c) shows, forgiving about the learning rate too.

**WHY BIAS CORRECTION IS NEEDED (THE $1-\beta^t$ TERMS)**

$m$ and $v$ both start at **zero**, so early on they are biased towards zero. At $t=1$, $m_1 = (1-\beta_1)g_1 = 0.1\, g_1$ — ten times too small. Dividing by $1-\beta_1^t$ undoes exactly this: at $t=1$ it divides by $0.1$, restoring $m_1 = g_1$, and the correction fades to 1 as $t$ grows. Without it the first few hundred steps are far too small, because with $\beta_2 = 0.999$ the term $1-\beta_2^t$ needs roughly a thousand steps to approach 1.

## 3.3 The code — what each cell does

| Cell | What it defines | What it is doing |
|---|---|---|
| code 12 | `MySGD`, `MyAdam` | Both subclass `torch.optim.Optimizer`. That means two obligations: pass the defaults to `super().__init__`, and loop over `self.param_groups` rather than a flat list, so per-group learning rates and schedulers work. Per-parameter state ($m$, $v$, the step count, the momentum buffer) lives in `self.state[p]`. |
| code 13 | `PureEMAMomentum` | The $(1-\mu)$ form, used to demonstrate the two conventions are equivalent. |
| code 14 | `compare_optimizers` | Trains two identical networks — same seed, same data, same order — one with my optimizer, one with PyTorch's, then compares every parameter. |
| code 15–16 | `trajectory` + contour plot | Runs each optimizer on the 2-D test function and records the path. |
| code 17–19 | `SmallCNN`, the sweep, the plots | Nine training runs: 3 optimizers × 3 learning rates. |

**TWO IMPLEMENTATION DETAILS THAT MUST MATCH PYTORCH EXACTLY**

**1.** The momentum buffer is initialised to **the gradient itself**, not to zero. Starting from zero would make the first step $\eta(1-\mu)g$ instead of $\eta g$.
**2.** Weight decay is folded into the gradient *first*, as $g \leftarrow g + \lambda\theta$. This is classic L2 and is what `torch.optim` does. It is **not** the same as AdamW's decoupled decay — apply it afterwards and the numbers disagree visibly.

### Result of part (a)

| Configuration | Max difference after 50 steps | Match? |
|---|---|---|
| SGD (plain) | 0.000e+00 | bit-for-bit identical |
| SGD + momentum 0.9 | 0.000e+00 | bit-for-bit identical |
| SGD + momentum + weight decay | 0.000e+00 | bit-for-bit identical |
| Adam | 4.2e−17 | yes |
| Adam, $\beta = (0.8, 0.99)$ | 5.6e−17 | yes |
| Adam + weight decay | 2.8e−17 | yes |

**WHY ADAM IS NOT BIT-IDENTICAL — AND WHY THAT IS FINE**

PyTorch computes the step as $\frac{\eta}{1-\beta_1^t} \cdot \frac{m}{\sqrt{v}/\sqrt{1-\beta_2^t}+\epsilon}$, while I compute $\eta \cdot \frac{m/(1-\beta_1^t)}{\sqrt{v/(1-\beta_2^t)}+\epsilon}$. These are the **same expression** algebraically, since $\sqrt{v/c} = \sqrt{v}/\sqrt{c}$ — but the operations happen in a different order, and floating-point arithmetic is not associative. A disagreement at the 17th digit confirms the formula is right. A disagreement at the 3rd would mean a real bug.

## 3.4 Part (b) — the narrow valley

The test function is

$$f(x,y) = \tfrac{1}{2}\left(x^2 + 50y^2\right) \qquad \Longrightarrow \qquad \nabla f = (x,\ 50y)$$

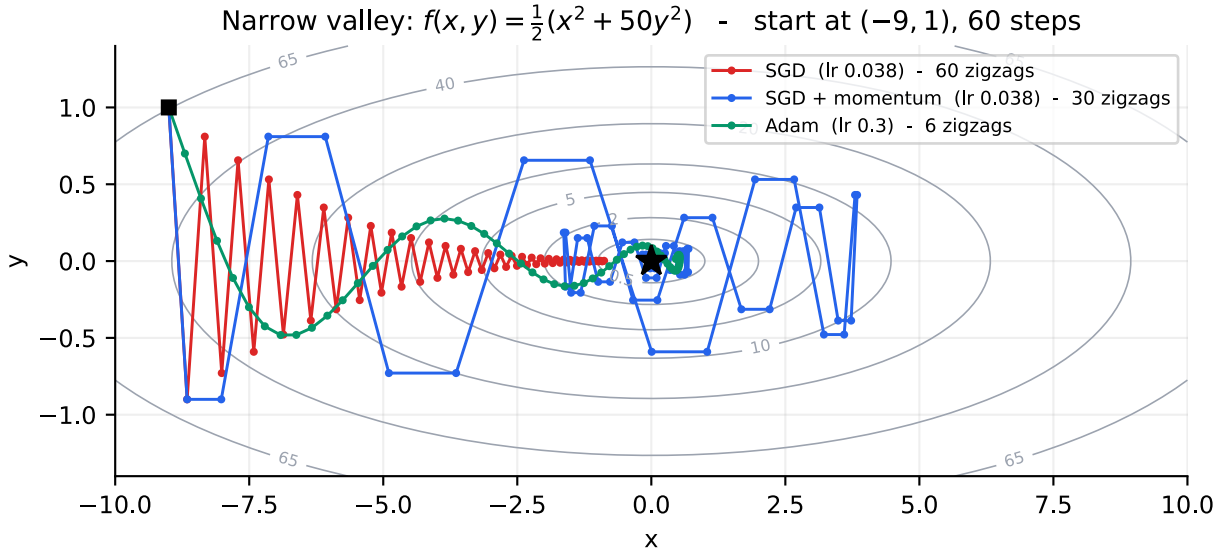
It is 50 times steeper in $y$ than in $x$. The contours are long thin ellipses forming a narrow valley along the $x$-axis, with the minimum at the origin. This is called an **ill-conditioned** problem, and the number 50 is its **condition number**.

**WHY ONE LEARNING RATE CANNOT SERVE BOTH DIRECTIONS**

Along $y$, gradient descent multiplies the error by $(1 - 50\eta)$ each step. That only converges while

$$|1 - 50\eta| < 1 \quad \Longleftrightarrow \quad \eta < \tfrac{2}{50} = 0.04$$

But along $x$ it multiplies by $(1 - \eta)$, so with $\eta$ forced below 0.04 progress along the valley is painfully slow. **The steep direction dictates the step size; the shallow direction is the one we actually need to travel.**




**Task 2(b).** All three start at $(-9, 1)$ and take 60 steps. SGD and momentum use $\eta = 0.038$, just under the stability limit — the fastest plain SGD can safely go. Adam uses $\eta = 0.3$, because its normalised step is about $\eta$ per iteration regardless of gradient size, so it must be sized to the *distance to travel*, not to the curvature.

| Optimizer | Sign flips in $y$ | Final $f$ | Reading |
|---|---|---|---|
| SGD | 60 of 60 | 0.388 | Bounces across the valley on **every single step** |
| SGD + momentum | 30 | 0.058 | Half the oscillation, and **6.7×** closer to the minimum |
| Adam | 6 | 0.109 | Almost no zigzag; takes a near-diagonal route |

**Reading the three behaviours:**

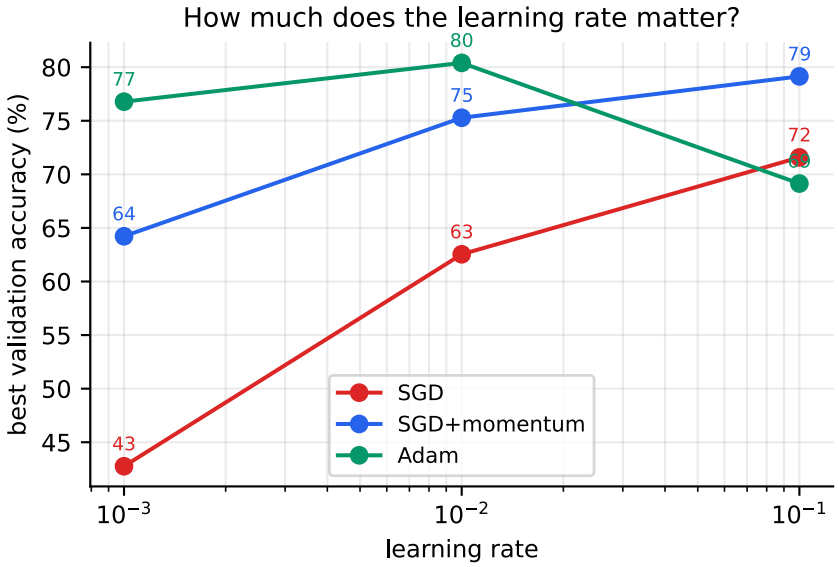
- **SGD zigzags.** With $\eta = 0.038$ the $y$-error is multiplied by $1 - 50(0.038) = -0.9$: the sign flips every step and the size only shrinks 10%. Meanwhile $x$ shrinks by $(1 - 0.038) = 0.962$ per step, so after 60 steps it has crawled from $-9$ to only $-0.88$. Nearly all the effort goes into the direction that does not matter.
- **Momentum damps it.** The $y$ component alternates sign so consecutive contributions cancel in the buffer; the $x$ component is consistent so it accumulates towards the tenfold amplification. Same learning rate, five times further along the valley.
- **Adam sidesteps the problem.** Dividing by $\sqrt{\hat{v}}$ normalises each coordinate by its own gradient history, so the huge $y$ gradients and small $x$ gradients produce steps of similar size. It effectively ignores the conditioning that cripples SGD.

**THE HONEST NUANCE**

The **best final value belongs to momentum, not Adam**. On a clean, noiseless, convex problem like this, well-tuned momentum is hard to beat. Adam's real advantage appears in part (c), where the gradients are noisy and badly scaled across layers.

## 3.5 Part (c) — how much does the learning rate matter?

The same grid $\eta \in \{0.001,\ 0.01,\ 0.1\}$ is used for **all three** optimizers. That is deliberate: if each were given its own favourable range, the plot would only show that all three can be made to work. With a shared grid you can read off *where* each one peaks and *how sharply* it falls away.




**Task 2(c).** SmallCNN, 141,034 parameters, 15 epochs on the full 45k training split. Adam peaks in the middle of the grid; both SGD variants are still climbing at the right edge.

| Optimizer | $\eta = 0.001$ | $\eta = 0.01$ | $\eta = 0.1$ | Spread |
|---|---|---|---|---|
| SGD | 42.8% | 62.5% | **71.6%** | **28.8 pts** |
| SGD + momentum | 64.2% | 75.3% | **79.1%** | 14.9 pts |
| Adam | 76.8% | **80.4%** | 69.1% | 11.3 pts |

**THE HEADLINE IS SENSITIVITY, NOT THE WINNER**

Plain SGD swings **28.8 points** across the grid — nearly useless at $\eta = 0.001$ (42.8%, barely better than the Task 1 MLP). Adam varies by only **3.6 points** between $\eta = 0.001$ and $\eta = 0.01$, both above 76%. **A careless learning rate costs almost nothing with Adam and nearly thirty points with plain SGD.** That is the practical reason Adam is the default when there is no time to tune.

**BUT ADAM IS THE ONLY ONE THAT ACTUALLY BREAKS**

It collapses from 80.4% to 69.1% at $\eta = 0.1$ — an 11-point drop — while both SGD variants are still improving there. Adam's step is about $\eta$ per parameter regardless of gradient size, so $\eta = 0.1$ means updates as large as the weights themselves, and training destabilises. The flatness that makes Adam forgiving at small rates is a liability at large ones.

**AN HONEST LIMITATION OF THIS GRID**

Both SGD curves are **still rising** at $\eta = 0.1$, so their optima lie at or beyond the right edge — this grid does not bracket them. Plain SGD would probably keep improving to about $\eta \approx 0.3$. So the 71.6% is a **lower bound**, not SGD's ceiling. Three points per optimizer is enough to rank sensitivity but not enough to locate the best rate.

**WHERE THE TENFOLD AMPLIFICATION SHOWS UP**

It is *not* visible as a shift in peak position, because the grid truncates both SGD curves at the same point. It shows instead at matched quality: **momentum at $\eta = 0.01$ scores 75.3%, beating plain SGD at $\eta = 0.1$ (71.6%)** — better results with one tenth of the learning rate, exactly the $\frac{1}{1-\mu} = 10$ factor predicted above. Momentum buys you roughly a decade of learning rate.

**THE PAYOFF WORTH QUOTING**

This CNN reaches **80.4%** with **141,034** parameters. The Task 1 MLP managed **50.6%** with **1,707,274** — twelve times as many parameters for thirty points *less* accuracy. That single comparison is the whole case for convolution: weight sharing and locality, not scale.

# 4. Task 3 — Regularization and shortcuts

**WHAT THE ASSIGNMENT ASKED**

**(a)** Train a small CNN with batch norm on **5k training images**. Compare: baseline, weight decay, dropout, flip+crop, cutout, mixup, and all combined (implement cutout and mixup yourself). Tabulate train accuracy, validation accuracy, and their gap; plot the curves.
**(b)** Build CIFAR-style plain and residual networks of depth $6n + 2$ for depths 8, 20 and 56 (three stages, channels 16/32/64, global average pooling, batch norm, Kaiming initialisation). The only difference between the two families is the shortcut. Plot the training loss versus epoch and explain.

**THE CRUCIAL FRAMING: THESE ARE TWO DIFFERENT FAILURES**

Part (a) is about **too little data** — the network memorises the training set and learns nothing general. Part (b) is about **too many layers** — the network cannot be trained at all, even on data it is actively being fitted to. They look similar from a distance ("the deep model is worse") but they have completely different causes and completely different cures. Being able to tell them apart is the point of this task.

## 4.1 Part (a) — six ways to stop a network memorising

The training set is deliberately cut to **5,000 images**, one ninth of the full split. Regularization only matters when data is scarce: with all 45,000 images a 141k-parameter CNN barely overfits, and the effect would be too small to measure.

### What each method does

| Method | Mathematics | In plain words |
|---|---|---|
| **weight decay** | $L' = L + \lambda\|w\|^2,\ \lambda = 5 \times 10^{-4}$ i.e. $g \leftarrow g + \lambda w$ | Adds a penalty for being large, pulling every weight gently towards zero. Simpler functions, less room to memorise. |
| **dropout** | $p = 0.3$ | Randomly switches off 30% of the features during training. The network cannot rely on any one feature always being present. |
| **flip + crop** | — | Randomly mirror the image left-right, and take a random 32×32 window from a 4-pixel-padded version. The network never sees the same framing twice. |
| **cutout** | zero an $8 \times 8$ square | Blanks out a random patch. If the net leans on one distinctive feature — a cat's ear — it fails whenever that feature is hidden, so it is forced to use the whole picture. |
| **mixup** | see below | Trains on *blends* of two images with blended labels. Forces sensible behaviour in the gaps between training examples. |

### The two I implemented from scratch

**Cutout** picks a random centre and zeroes an $8 \times 8$ square. The zeroing happens *after* normalisation, so the hole takes the per-channel mean colour rather than black.

**Mixup** blends two images and their labels in the same proportion:

$$\tilde{x} = \lambda x_i + (1-\lambda) x_j \qquad L = \lambda\,\mathrm{CE}\big(f(\tilde{x}), y_i\big) + (1-\lambda)\,\mathrm{CE}\big(f(\tilde{x}), y_j\big)$$

with $\lambda \sim \mathrm{Beta}(\alpha, \alpha)$ and $\alpha = 1$ (which makes $\lambda$ uniform on $[0, 1]$).




**What the augmentations actually do** to one CIFAR-10 image. Flipping and cropping produce images that are still obviously the same object — they add *real* information about what does not change. Cutout hides evidence. Mixup produces an image that does not exist at all, labelled 60% one class and 40% the other.

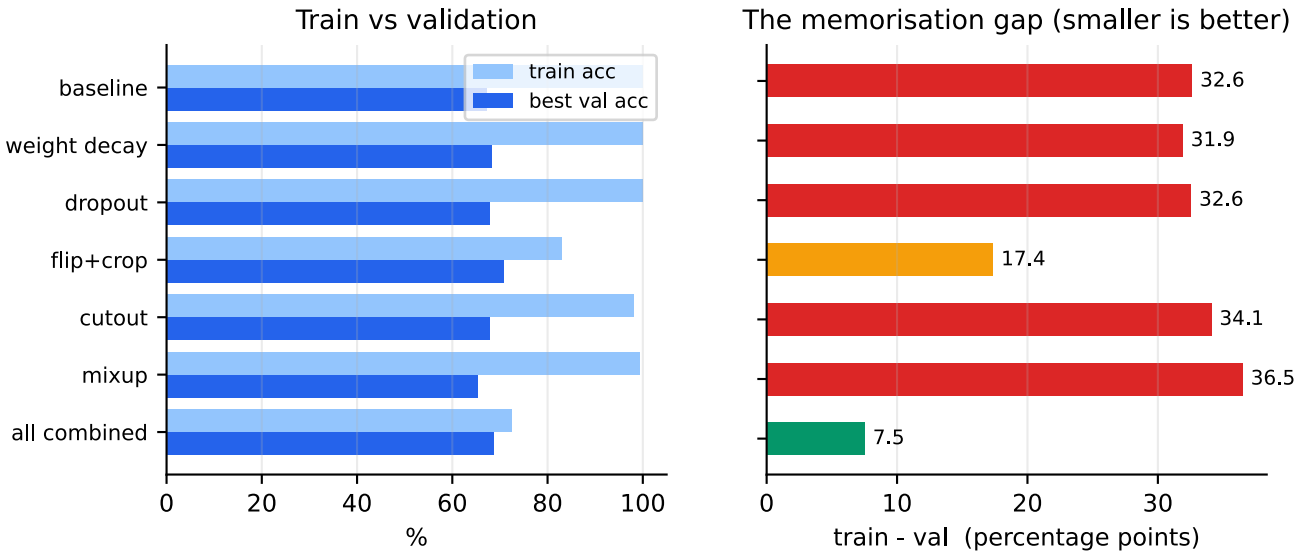
**WHY MIXUP IS NOT AS STRANGE AS IT LOOKS**

Asking a network to predict "60% cat, 40% ship" sounds absurd. The purpose is to force the network to behave **linearly between training examples**. Left alone, a network makes wildly overconfident predictions in the empty space between the examples it has seen — and that over-confidence *is* the memorisation. Mixup fills that space with sensible targets.

**A MEASUREMENT DECISION THAT MATTERS**

Training accuracy is always measured on **clean, un-augmented images**, in a separate pass. If it were measured on augmented batches, mixup's "training accuracy" would be computed against blended labels — a different quantity entirely — and the gap column would not be comparable across rows. The gap is the whole point of the table, so it has to measure the same thing everywhere.

## What I measured




**Task 3(a).** Left: training accuracy (pale) against best validation accuracy (solid). Right: the memorisation gap. Red bars are the configurations that still memorise completely; green is the one that does not.

| Config | Train acc | Best val acc | Gap | Reading |
|---|---|---|---|---|
| baseline | 100.00% | 67.36% | 32.6 | Perfect memorisation of all 5,000 images |
| weight decay | 100.00% | 68.42% | 31.9 | +1.1 pts — still memorises completely |
| dropout | 100.00% | 67.80% | 32.6 | +0.4 pts — almost no effect |
| **flip+crop** | 83.10% | **70.88%** | 17.4 | **Best single method, +3.5 pts** |
| cutout | 98.06% | 67.94% | 34.1 | +0.6 pts |
| mixup | 99.46% | 65.46% | 36.5 | −1.9 pts — *hurt* at this scale |
| all combined | 72.56% | 68.82% | **7.5** | Smallest gap by far, +1.5 pts |

**THE CLEAREST FINDING: ONLY AUGMENTATION ACTUALLY STOPS MEMORISATION**

Look at the training-accuracy column. Baseline, weight decay and dropout **all reach exactly 100%** — they memorise all 5,000 images perfectly despite the regularization. Only the methods that **change the data** (flip+crop, and the combination) prevent that, because the network never sees the same image twice and so has nothing fixed to memorise.

So the two mechanisms are not equally effective here: **constraining the weights barely helped; changing the data did.**

**WHY DROPOUT DID ALMOST NOTHING — WORTH BEING ABLE TO EXPLAIN**

In my `SmallCNN`, dropout sits **only after global average pooling**, on a 128-dimensional vector just before the classifier. That is a much weaker intervention than dropout between convolutional layers. The result is a property of *where it was placed*, not evidence that dropout is useless.

Similarly, weight decay is applied to **every** parameter including the batch-norm scale and shift. Common practice excludes those. Both choices are defensible, but they explain the small effects.

**TWO RESULTS THAT LOOK LIKE FAILURES BUT ARE FINDINGS**

**Mixup hurt (−1.9 points).** With $\alpha = 1$ on only 5,000 images, mixup is very strong — nearly every training image is a blend, and the network gets little clean signal. Mixup usually needs more data or longer training to pay off.

**"All combined" is *not* the best.** It has by far the smallest gap (7.5 points) but only 68.82% validation — below flip+crop alone at 70.88%. Five regularizers on 5,000 images **over-constrain** a small network: training accuracy falls to 72.56%, which is **underfitting**, not successful regularization. More regularization is not automatically better.

**THE 30-SECOND VERSION OF PART (A)**

"With only 5,000 images my network memorised them perfectly — 100% on training, 67% on validation. Penalising the weights barely helped; it still memorised. What worked was changing the pictures: flipping and cropping them so it never saw the same image twice. That gave the best result. And stacking every technique at once actually made it worse — it closed the gap, but by making the network too constrained to learn."

## 4.2 Part (b) — the degradation problem

These are the CIFAR networks from the ResNet paper: one stem convolution, then **three stages** of $n$ blocks each at **16, 32 and 64 channels**, then global average pooling and a linear classifier. Each block has two $3 \times 3$ convolutions, so total depth is

$$\text{depth} = 6n + 2 \qquad n = 1 \Rightarrow 8, \quad n = 3 \Rightarrow 20, \quad n = 9 \Rightarrow 56$$

### The only difference between the two families

```
PLAIN BLOCK                          RESIDUAL BLOCK

  x                                    x ----------------+
  |                                    |                 |
 conv 3x3                            conv 3x3           |  shortcut
 batchnorm                           batchnorm          |  (identity)
 ReLU                                ReLU               |
  |                                    |                 |
 conv 3x3                            conv 3x3           |
 batchnorm                           batchnorm          |
  |                                    |                 |
  |                                    + <---------------+
  |                                    |
 ReLU                                ReLU
  |                                    |
 out                                 out

 out = F(x)                          out = F(x) + x
```

In code it is literally one line:

```
if self.residual:
    out = out + self.shortcut(x)
```

**KEEPING THE COMPARISON HONEST: OPTION A SHORTCUTS**

Where a block changes shape, the shortcut has to match. I used **Option A** from the ResNet paper — subsample with a stride and zero-pad the extra channels — rather than a $1 \times 1$ convolution. Option A has **no parameters at all**, so:

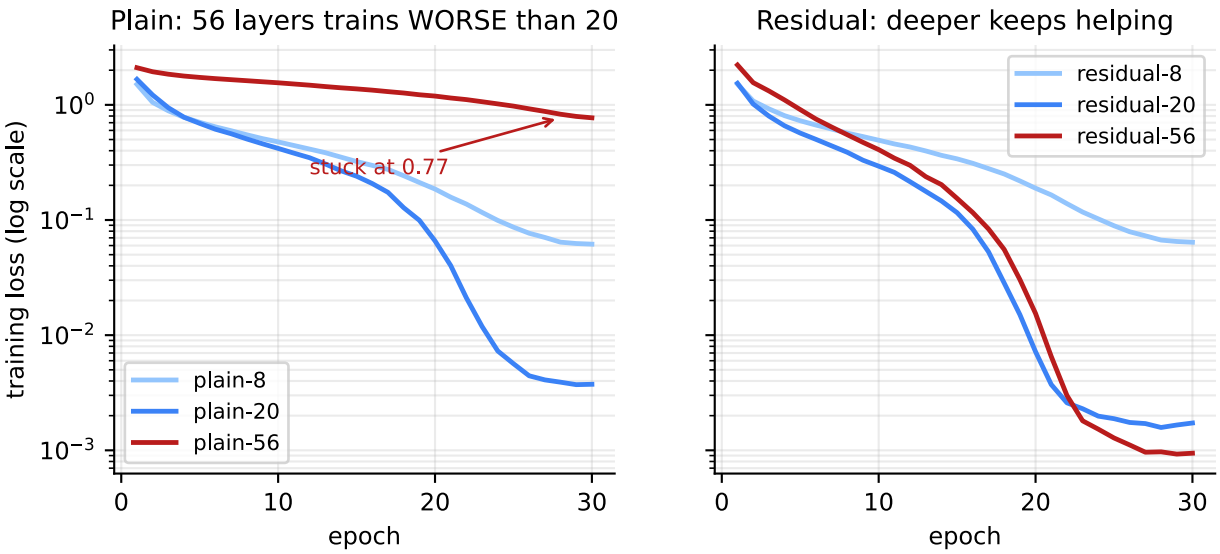
| Depth | Plain | Residual | Identical? |
|---|---|---|---|
| 8 | 75,290 | 75,290 | yes |
| 20 | 269,722 | 269,722 | yes |
| 56 | 853,018 | 853,018 | yes |

The two families have **exactly** the same number of parameters at every depth. So any difference in the plot below *cannot* be explained by one having more capacity. It can only be the shortcut.

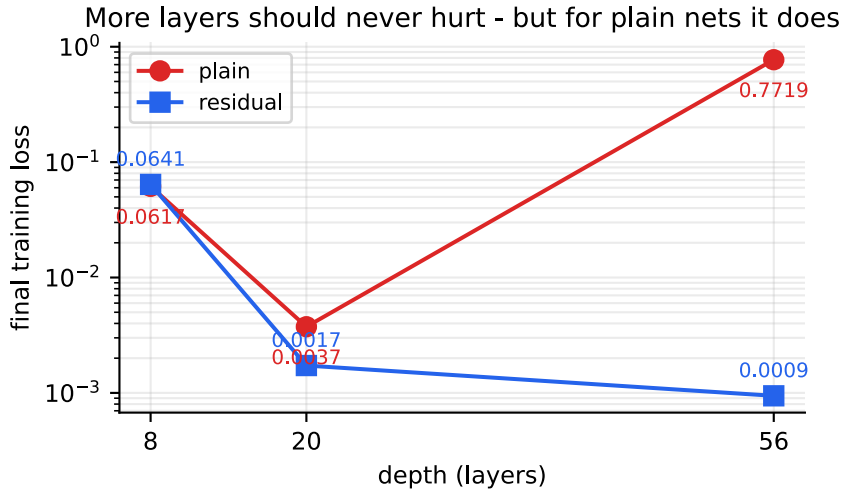
**ALSO: NO DATA AUGMENTATION IN PART (B)**

Unlike part (a). The question here is whether a deep network can be **optimised**, so the networks are allowed to fit the training data as hard as they can, and the training-loss curves are compared directly. Augmentation would damp exactly the effect being measured.

## What I measured




**Task 3(b).** Training loss on a logarithmic scale, 30 epochs. Left: the plain family. The 56-layer network (dark red) gets **stuck** — it ends far worse than its own 20-layer version. Right: the residual family. Depth keeps helping.




The same result as one picture: final training loss against depth. For the residual family, deeper is monotonically better. For the plain family, going from 20 to 56 layers makes training loss **200 times worse**.

| Network | Parameters | Final training loss | Validation accuracy |
|---|---|---|---|
| plain-8 | 75,290 | 0.0617 | 79.40% |
| residual-8 | 75,290 | 0.0641 | 79.10% |
| plain-20 | 269,722 | 0.0037 | 82.56% |
| residual-20 | 269,722 | 0.0017 | 83.16% |
| **plain-56** | 853,018 | **0.7719** | **57.72%** |
| **residual-56** | 853,018 | **0.0009** | **80.78%** |

#### THE HEADLINE NUMBERS

- **plain-56 trains 200× worse than plain-20** (0.7719 vs 0.0037) despite having **three times more parameters**.
- **residual-56 trains 860× better than plain-56** (0.0009 vs 0.7719) with **exactly the same parameter count**.
- At depth 8 the two are **effectively tied** (0.0617 vs 0.0641).

#### WHY THIS IS NOT OVERFITTING — THE SINGLE MOST IMPORTANT POINT

Overfitting means **low** training loss with poor validation accuracy. Here the **training** loss itself is bad — on data the network is actively being fitted to. That is a different disease entirely.

And it should be impossible. A 56-layer network can represent *anything* a 20-layer one can: just set the extra 36 layers to the identity function and you recover the shallower network exactly. So plain-56 has **strictly more capacity** and *must* be able to reach at least as low a training loss as plain-20.

It fails to. So the capacity is there and **gradient descent cannot find it**. This is an **optimisation** failure, and the ResNet paper named it the **degradation problem**.

#### WHY THE SHORTCUT FIXES IT

A residual block computes $\mathbf{out} = \boldsymbol{F}(\boldsymbol{x}) + \boldsymbol{x}$. To represent the identity it only needs to drive $\boldsymbol{F}$ to **zero** — and driving a thing to zero is the easiest thing a network with weight decay does. A plain block must *learn* to reproduce its own input before it can improve on it, and 56 layers of that is hard.

The shortcut also gives the gradient a path with derivative **exactly 1** all the way back to earlier layers — the clean route that Task 1(d) showed sigmoid lacked. ReLU and Kaiming initialisation fixed the signal *at initialisation*; at 56 layers it degrades again *during training*, and the shortcut is what holds the path open.

#### READING THE DEPTH-8 TIE CORRECTLY

At 8 layers there is no degradation to fix, so the shortcut buys nothing — plain-8 even edges ahead by a hair (0.0617 vs 0.0641). **This is a feature of the result, not noise to explain away:** it shows the benefit is specifically about *depth*, not a free improvement you get everywhere.

Also note residual-56 (0.0009) beat residual-20 (0.0017) on training loss but *not* on validation accuracy (80.78% vs 83.16%). With only 30 epochs and no augmentation, the 56-layer network fits the training set hardest and generalises slightly worse. The ResNet paper trained for 180 epochs with augmentation.

**THE 30-SECOND VERSION OF PART (B)**

"I built the same network twice, plain and with shortcut connections, with *identical* parameter counts. At 8 and 20 layers they tie. At 56 layers the plain one breaks — its training loss is 200 times worse than its own 20-layer version, even though it has three times more parameters. That is not overfitting, because the *training* loss is what got worse. The deeper network could copy the shallower one by setting layers to the identity, so the ability is there and gradient descent just cannot find it. Adding one line — `out = out + x` — fixes it completely: the residual version reaches 0.0009 instead of 0.77."

# 5. Task 4 — Best network under 500k parameters

**WHAT THE ASSIGNMENT ASKED**

Design the best CNN you can with **at most 500k parameters** and **at most 20 training epochs**. Tune on the validation split. Report an ablation of at least three design choices (e.g. shortcut, augmentation, weight decay, optimizer), and the test accuracy of your final model, **evaluated once**.

## 5.1 Designing to the budget

Twenty epochs is the binding constraint, and it changes what the right architecture is. A very deep network spends most of a short schedule just getting organised. Task 3(b) showed this directly: at 30 epochs the 56-layer residual network had *not* caught up with the 20-layer one on validation accuracy (80.78% vs 83.16%).

**THE DESIGN DECISION, IN ONE SENTENCE**

Start from **depth 20** — the shape that actually worked in Task 3(b) — and spend the remaining parameter budget on **width** rather than more layers.

Scaling the channels up from the standard 16/32/64:

| Configuration | Depth | Parameters | Fits 500k? |
|---|---|---|---|
| 16 / 32 / 64 (Task 3b baseline) | 20 | 269,722 | yes, but wastes 46% of the budget |
| **21 / 42 / 84** | **20** | **463,627** | **yes — 92.7% of budget** |
| 22 / 44 / 88 | 20 | 508,672 | no, over by 8,672 |
| 24 / 48 / 96 | 14 | 386,626 | yes, but shallower |
| 20 / 40 / 80 | 26 | 572,430 | no |

**21/42/84 at depth 20** is the choice: the widest version of the known-good shape that still fits. The odd channel counts are simply what the budget permits.

## 5.2 The recipe, and where each ingredient came from

| Ingredient | Setting | Evidence |
|---|---|---|
| architecture | residual, depth 20, widths 21/42/84 | Task 3(b): shortcuts win, and depth 20 generalised best |
| optimizer | SGD, momentum 0.9 | Task 2(c): best accuracy at a well-chosen rate |
| learning rate | 0.1, cosine annealed to 0 | Task 2(c): SGD wanted the top of the grid |
| weight decay | $5 \times 10^{-4}$ | Task 3(a) |
| augmentation | flip + pad-4 crop + cutout(8) | Task 3(a): flip+crop was the only method that actually stopped memorisation |
| batch size | 128 | used throughout |

Cosine annealing is the one ingredient not inherited from an earlier task. The learning rate follows

$$\eta_t = \frac{\eta_0}{2}\left(1 + \cos\frac{\pi t}{T}\right)$$

decaying smoothly from $\eta_0 = 0.1$ to zero over $T = 20$ epochs. With such a short budget there is no room for a step schedule's long plateaus, and annealing smoothly extracts most of what a longer schedule would give: large steps early to travel far, tiny steps late to settle.

## 5.3 The ablation

Each row removes **exactly one** ingredient and changes nothing else, so the difference is attributable to that ingredient alone.

| Configuration | What changed | Best val | Change |
|---|---|---|---|
| **full recipe** | — | **89.60%** | baseline |
| **minus augment** | no flip, crop or cutout | 85.48% | **−4.12 pts** |
| Adam instead | Adam at $\eta = 10^{-3}$ | 86.90% | −2.70 pts |
| minus shortcut | plain network, **identical** parameter count | 87.56% | −2.04 pts |
| minus weight decay | $\lambda = 0$ | 87.58% | −2.02 pts |

**AUGMENTATION WINS BY A FACTOR OF TWO — AND THE TRAINING LOSS SHOWS WHY**

Removing augmentation costs **4.12 points**; the next-largest removal costs 2.70. The reason is visible in the training loss: without augmentation it falls to **0.0047**, about **48 times lower** than the full recipe, while validation accuracy drops four points. That is the textbook signature of overfitting — fitting the training data far *better* and the held-out data far *worse*.

| Configuration | Final training loss | Mechanism |
|---|---|---|
| full recipe | 0.2281 | — |
| minus augment | 0.0047 | **generalisation** — fits training data much harder |
| minus shortcut | 0.3228 | **optimisation** — fits training data *less* well |
| minus weight decay | 0.2848 | mild generalisation cost |
| Adam instead | 0.3284 | optimises less effectively here |

**THE SHORTCUT ACTS THROUGH A DIFFERENT MECHANISM ENTIRELY**

Removing it makes the training loss go **up** (0.2281 → 0.3228) — the plain network fits the training data *less* well, not more. So this is an **optimisation** cost, exactly the distinction Task 3(b) established, and not a generalisation one. Both have identical parameter counts (463,627). The effect is smaller here than at depth 56 because depth 20 is comfortably trainable without shortcuts.

**SHORTCUT AND WEIGHT DECAY ARE TIED, NOT RANKED**

They differ by **0.02 points** (−2.04 vs −2.02) — far below the few tenths of a point of run-to-run variation from a single seed. Reporting one as more important than the other would be reading noise.

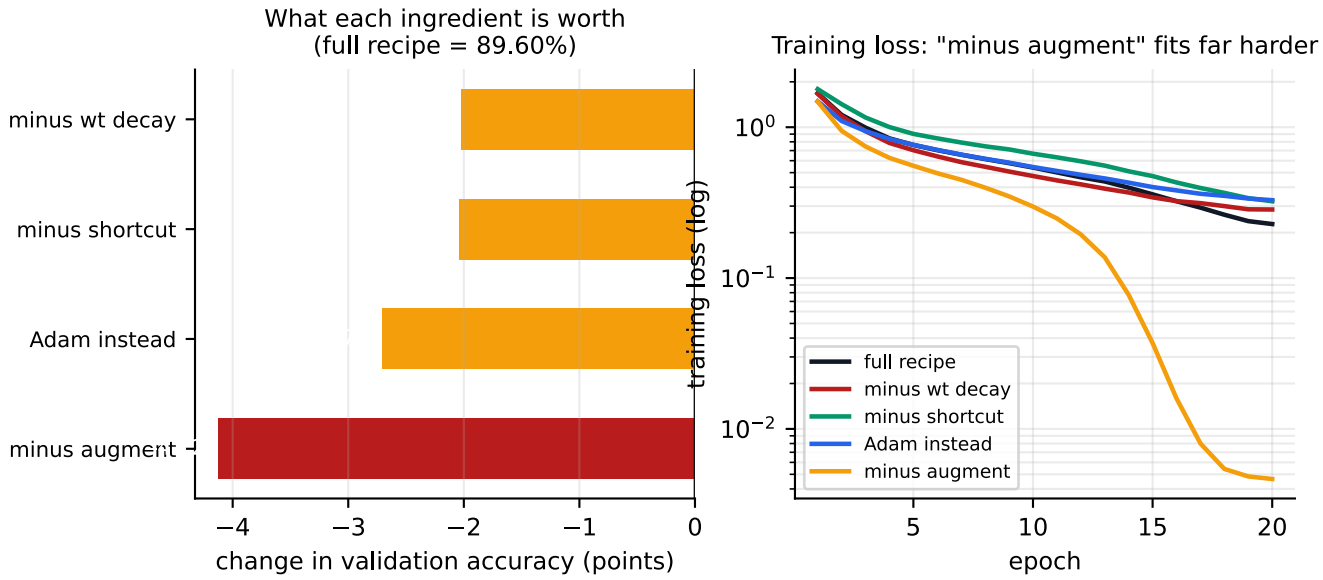
**SGD BEAT ADAM HERE, REVERSING TASK 2(C)**

Adam won on the small CNN at 15 epochs; here SGD with cosine annealing wins by 2.70 points. Adam's advantage is being *forgiving* about the learning rate — but once the rate is well chosen and annealed smoothly to zero, tuned momentum overtakes it. Adam also ends at a higher training loss (0.3284), so it is genuinely optimising less effectively, not merely generalising differently.

**TWO CAVEATS TO STATE WHEN REPORTING IT**

**Single seed.** Run-to-run variation on CIFAR-10 at this scale is a few tenths of a point, so any difference smaller than roughly half a point should be read as "no measurable effect", not a real ranking.

**No interactions.** A one-factor-at-a-time ablation cannot see interactions — removing two ingredients need not cost the sum of removing each. Task 3(a) showed exactly this: every regularizer helped alone, but all five together were *worse* than flip+crop by itself.




**Task 4 ablation.** Left: what each ingredient is worth, measured by removing it. Right: training loss on a log scale. Note the amber curve ("minus augment") plunging far below the others — it fits the training data 48 times harder and generalises four points worse. That divergence *is* overfitting, caught in the act.

## 5.4 The single test evaluation

**WHY "ONCE" IS ENFORCED BY CODE, NOT BY INTENTION**

Every time a number from the test set influences a decision, it stops being a held-out measure of generalisation and quietly becomes part of the training procedure. Had I run the ablation against the test set, the winning configuration would have been *selected* using test data and the reported accuracy would be optimistic by an unknown amount.

So the notebook evaluates it in one dedicated cell and caches the answer to `results/t4_test_eval.json`. Re-running the notebook reads the stored value instead of measuring again. Deleting that file is the only way to repeat the measurement — which is exactly the moment you would be contaminating the estimate.

| Final model | |
|---|---|
| architecture | residual, depth 20, widths 21/42/84 |
| parameters | 463,627 of 500,000 allowed (92.7%) |
| epochs | 20 of 20 allowed |
| validation accuracy | 89.60% |
| **test accuracy** | **90.05%** |

**TEST CAME IN 0.45 POINTS ABOVE VALIDATION**

90.05% against 89.60% — the comfortable outcome. The validation split absorbed every design decision in this task, so it carries some optimistic bias, but with only five configurations compared that bias is evidently smaller than the noise between the two splits. The test set is also **twice the size** of the validation set (10,000 vs 5,000), making it the less noisy measurement of the two. Had test come in several points *below* validation, that gap would have been the price of tuning, and some of the winning margin would have been luck.

**THE BINDING CONSTRAINT WAS EPOCHS, NOT PARAMETERS**

The full recipe ends at a training loss of 0.2281 — nowhere near zero — and its validation curve is still rising at epoch 20. The model is **underfitted**, not saturated. Published CIFAR-10 ResNets reach 93–95%, but they train for 180+ epochs, nine times this budget. Given more, the order to try things is: **longer training first** (the curve says there is more to extract), **then more width** (36,373 parameters are still unused), and **more depth last** — because Task 3(b) showed depth does not pay without the epochs to support it.

# 6. Explaining this to someone in five minutes

If you have to summarise the whole assignment out loud, this is the order that works. Each step sets up the next.

**1. START WITH THE PROBLEM (20 SECONDS)**

"CIFAR-10 is 60,000 tiny photos in 10 categories. I had to build networks to classify them — starting from writing the learning algorithm itself by hand, with no automatic differentiation."

**2. TASK 1 — HOW LEARNING WORKS (60 SECONDS)**

"I implemented the forward and backward pass of every layer myself. The nice result is that softmax combined with cross-entropy has the gradient *predicted probability minus true probability* — all the messy terms cancel.

I proved it two ways: against PyTorch's autograd, which matched to 16 decimal places, and by nudging individual weights and measuring the slope directly, which matched to 9.

Then I trained it. It got **50.6%** — and that is a ceiling, not a bug. Flattening an image into a list of 3072 numbers throws away the fact that neighbouring pixels are related, and that a cat shifted three pixels right is still a cat."

**3. TASK 2 — HOW YOU DESCEND MATTERS (60 SECONDS)**

"Same network, different optimizers. On a deliberately stretched valley, plain gradient descent zigzags across it on *every single step* — 60 out of 60 — because the step size is dictated by the steep direction while the direction you need to travel is the shallow one. Momentum, which accumulates a velocity, halves the zigzag. Adam, which divides each coordinate by its own gradient history, almost eliminates it.

On the real CNN, the learning rate alone moved accuracy from **43% to 80%**. The headline is *sensitivity*: plain SGD swings 29 points across the grid, Adam only 11. That is why Adam is the default when you cannot afford to tune."

**4. TASK 3 — TWO DIFFERENT FAILURES (90 SECONDS)**

"Networks fail in two ways that look alike but are not.

**Too little data.** On 5,000 images mine hit 100% on training and 67% on validation — perfect memorisation. Penalising the weights barely helped; it still memorised. What worked was changing the *pictures*: flipping and cropping so it never saw the same image twice.

**Too many layers.** I built the same network plain and with shortcut connections, with *identical* parameter counts. At 56 layers the plain one broke — training loss 200 times worse than its own 20-layer version, despite three times the parameters. That is not overfitting, because the *training* loss is what got worse. A 56-layer net could copy a 20-layer one by setting the extra layers to the identity, so the ability is there and gradient descent just cannot find it. One line — `out = out + x` — fixes it: 0.0009 instead of 0.77."

**5. TASK 4 AND THE PUNCHLINE (30 SECONDS)**

"Finally I combined everything under a budget: at most 500k parameters and 20 epochs. I spent the budget on width rather than depth, because Task 3 showed depth needs more epochs than I had. It scored **90.05% on the test set**, which I measured exactly once.

The ablation says augmentation mattered most by a factor of two — and you can see why in the training loss: without it the network fits the training data 48 times harder and the held-out data four points worse.

The thread through all of it is that **structure beats size**. My best model has fewer parameters than the fully-connected network I started with, and is far more accurate — because convolution, shortcuts and augmentation each build in something true about images that the first network had to try to learn from data, and could not."

## 6.1 The five questions you are most likely to be asked

| Question | Short answer |
|---|---|
| **Why is softmax fused with cross-entropy?** | Because the gradient simplifies to $\boldsymbol{p} - \boldsymbol{y}$ — the softmax Jacobian cancels against the logarithm. Separately they are both messier and numerically unstable. |
| **Why does Kaiming initialisation use 2/fan-in?** | Because ReLU switches off about half the units, costing a factor of 2 in signal variance per layer. The 2 in the numerator is exactly that correction. It is derived *for* ReLU. |
| **Why no $(1-\mu)$ in your momentum?** | Both conventions exist and are the same algorithm — they differ by a constant $(1-\mu)$ absorbed into the learning rate. I used PyTorch's form because the task required matching `torch.optim.SGD`. My Adam *does* use the $(1-\beta)$ form, because Adam is defined that way. |
| **Is plain-56 failing because it overfits?** | No. Overfitting means *low* training loss with poor validation. Here the training loss itself is bad, on data it is being fitted to. It is an optimisation failure. |
| **Why does the test set only get used once?** | Because any decision made by looking at it turns it into training data, and the reported accuracy stops being an honest estimate of generalisation. |

# 7. Symbol glossary

| Symbol | Means | Where it appears |
|---|---|---|
| $L$ | the loss — how wrong the network is | everywhere |
| $g$ or $g_t$ | gradient, $\partial L / \partial(\text{something})$ | Tasks 1, 2 |
| $W$, $b$ | weight matrix, bias vector | Task 1(a) |
| $x$, $y$, $z$ | input, true label, logits (raw scores) | Task 1 |
| $N$, $C$ | batch size, number of classes (10 here) | Task 1 |
| $\eta$ (eta) | learning rate — how big a step to take | Task 2 |
| $\mu$ (mu) | momentum coefficient, usually 0.9 | Task 2 |
| $\beta_1, \beta_2$ | Adam's two decay rates, 0.9 and 0.999 | Task 2 |
| $m_t$, $v_t$ | Adam's running mean and running variance of the gradient | Task 2 |
| $\epsilon$ (epsilon) | tiny constant, $10^{-8}$, stops division by zero | Task 2 |
| $\varepsilon$ | the nudge size in finite differences, $10^{-5}$ | Task 1(b) |
| $\lambda$ (lambda) | weight-decay strength; also mixup's blend ratio | Tasks 3, 4 |
| $\sigma$ (sigma) | the sigmoid function; also a standard deviation | Task 1(d) |
| $\mathbb{1}[\cdot]$ | 1 if the condition holds, 0 otherwise | Task 1(a) |
| $\odot$ | multiply element by element | Task 1(a) |
| $n$ | blocks per stage; depth is $6n + 2$ | Task 3(b) |
| $F(x)$ | what a residual block computes before adding $x$ back | Task 3(b) |

## 7.1 Jargon, decoded

| Term | Plain meaning |
|---|---|
| **epoch** | One complete pass through all the training images. |
| **batch** | A small group of images (128 here) processed together before updating the weights. |
| **logits** | The raw scores the network outputs, before they are turned into probabilities. |
| **autograd** | PyTorch's automatic differentiation — the thing Task 1 forbids you from using. |
| **batch norm** | A layer that re-centres and re-scales its inputs using the statistics of the current batch. Makes training much more stable. |
| **global average pooling** | Replace each channel's whole feature map with its average value. Turns a $64 \times 4 \times 4$ block into just 64 numbers, cheaply. |
| **ill-conditioned** | Much steeper in one direction than another, so a single step size cannot suit both. |
| **generalisation gap** | Training accuracy minus validation accuracy. How much of what the network knows is memorisation. |
| **ablation** | Removing one ingredient at a time to find out how much each contributes. |

---

Companion to `EN4554_a02.ipynb` — Lakshan R.G.R, 220355M. All figures and numbers are generated from the measured results in `results/*.json`. Final model: 463,627 parameters, 20 epochs, **90.05% test accuracy**.